CANDIDATE'S DECLARATION

I hereby declare that the project work entitled **SmartInvest — Mutual Fund Portfolio Manager** submitted to **Gujarat Technological University** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Engineering in Information Technology (Semester VIII)** is a record of original work done by me under the supervision and guidance of the faculty guide.

The project has not previously formed the basis for the award of any Degree, Diploma, Associateship, Fellowship, or other similar titles. All the information furnished in this report is true and complete to the best of my knowledge and belief.

Name: Tirth Dadhaniya

Enrollment No.: _____

Branch: Information Technology

Semester: VIII

Institution: _____

Date: _____

Signature of Candidate: _____

ACKNOWLEDGEMENT

The completion of this project has been made possible through the valuable guidance, support, and encouragement of several individuals to whom I owe a debt of gratitude.

I express my sincere gratitude to **Gujarat Technological University** for prescribing this project as a mandatory component of the B.E. (IT) curriculum, thereby providing an opportunity to apply academic knowledge to real-world engineering problems.

I am deeply thankful to my **Faculty Guide** for providing continuous technical mentorship, reviewing the progress of the project at each stage, and offering constructive feedback that improved both the engineering quality and the academic rigor of this report. The suggestions provided during the project review sessions shaped the architecture decisions and the validation strategy adopted in SmartInvest.

I extend heartfelt thanks to the **Head of Department, Information Technology**, and the **Principal** of the institute for creating an environment conducive to technical learning and project development.

I also acknowledge the open-source community and the maintainers of the **MFAPI.in** platform, whose freely accessible Net Asset Value (NAV) data API made real-time mutual fund tracking feasible without commercial data licensing constraints.

Finally, I thank my family and friends for their moral support throughout the duration of this project.

Tirth Dadhaniya

B.E. Information Technology, Semester VIII

ABSTRACT

SmartInvest is a production-grade, full-stack Mutual Fund Portfolio Manager built using the MERN technology stack — MongoDB, Express.js, React 18, and Node.js. The application targets Indian retail investors who require a consolidated, data-driven platform to track lump-sum investments and Systematic Investment Plans (SIPs), monitor real-time portfolio performance using live Net Asset Value (NAV) data, and make goal-oriented financial decisions.

The system is architecturally divided into a React–Vite frontend and a Node.js–Express backend communicating over a RESTful API. Authentication is implemented using JSON Web Tokens (JWT) delivered through HTTP-only cookies, ensuring protection against Cross-Site Scripting (XSS) attacks. All incoming request payloads are validated through a centralised Zod schema layer, providing strict type safety at the API boundary. MongoDB Atlas serves as the persistent data store, with Mongoose ODM schemas enforcing domain-level data integrity for Users, Investments, SIPs, and Financial Goals.

The backend exposes more than thirty API endpoints covering user profile management, mutual fund investment creation, SIP lifecycle management, financial goal tracking, and a suite of analytical functions including portfolio health scoring, asset allocation computation, Expense Drain Analysis, LTCG tax estimation, break-even NAV calculation, What-If simulations, and stress testing. Live NAV data is sourced from the free MFAPI.in REST API, which provides scheme-level historical NAV series for over ten thousand Indian mutual fund schemes.

On the frontend, the application employs Tailwind CSS for a utility-first responsive design, Recharts for interactive financial data visualisations, and a custom skeleton-loading system that mirrors the production UI during asynchronous data fetches. A global AuthContext and PortfolioContext manage application state without third-party state management libraries, simplifying the component tree.

The financial computation engine within the backend services layer implements Weighted Average Cost (WAC) basis calculations for partial redemptions, compound annual growth rate (CAGR) derivation, SIP next-due-date projection, and scenario-based stress testing using configurable market-correction multipliers. The portfolio health score is a proprietary 0–100 composite metric assessing fund diversification, risk-category alignment, and expense ratio efficiency.

This report documents the complete software engineering lifecycle of SmartInvest: requirement elicitation, system analysis, architectural design, database schema design, API design, frontend component architecture, implementation specifics, and testing strategy.

LIST OF FIGURES

Figure No.	Caption	Page
Fig 1.1	SmartInvest High-Level System Architecture Diagram	5
Fig 2.1	Organisation Structure of SmartInvest Project Team	10
Fig 3.1	Use Case Diagram — Investor (Primary Actor)	14
Fig 3.2	Project Gantt Chart — Semester VIII Development Schedule	17
Fig 4.1	Activity Diagram — Investment Buy/Sell Workflow	22
Fig 4.2	Activity Diagram — SIP Lifecycle State Machine	25
Fig 4.3	Data Flow Diagram (Level 0) — SmartInvest System	27
Fig 5.1	Entity–Relationship Diagram — MongoDB Collections	32
Fig 5.2	State Transition Diagram — SIP Status Transitions	36
Fig 5.3	Sequence Diagram — NAV Fetch and Portfolio Refresh	38
Fig 6.1	Backend Folder Structure Tree	41
Fig 6.2	Frontend Component Hierarchy Diagram	45
Fig 6.3	Dashboard Screenshot — Portfolio Overview	48
Fig 6.4	Recharts Pie Chart — Asset Allocation Visualisation	50
Fig 6.5	Recharts Line Chart — NAV History over 12 Months	51
Fig 7.1	Test Case Execution Summary — Pass/Fail Distribution	56

LIST OF TABLES

Table No.	Caption	Page
Table 1.1	Technology Stack Summary	6
Table 2.1	Project Team Roles and Responsibilities	11
Table 3.1	Functional Requirements	13
Table 3.2	Non-Functional Requirements	14
Table 3.3	Technology and Library Review	16
Table 3.4	Effort and Time Estimation	18
Table 4.1	Current System Problem Analysis	21
Table 4.2	Feasibility Analysis Matrix	23
Table 4.3	Comparison of NAV Data Providers	26
Table 5.1	User Schema — Fields and Constraints	31
Table 5.2	Investment Schema — Fields and Constraints	33
Table 5.3	SIP Schema — Fields and Constraints	34
Table 5.4	Goal Schema — Fields and Constraints	35
Table 5.5	API Endpoint Reference — Authentication Module	39
Table 5.6	API Endpoint Reference — Investment Module	40
Table 5.7	API Endpoint Reference — SIP Module	41
Table 5.8	API Endpoint Reference — Goals Module	42
Table 5.9	API Endpoint Reference — Analytics Module	43
Table 6.1	Zod Validation Schema — Investment Payload	44
Table 6.2	Portfolio Health Score Computation Criteria	47
Table 6.3	Stress Testing Scenario Parameters	52

Table No.	Caption	Page
Table 7.1	Test Cases — Authentication Module	55
Table 7.2	Test Cases — Investment CRUD Operations	57
Table 7.3	Test Cases — SIP Lifecycle Operations	58
Table 7.4	Test Cases — Portfolio Analytics Endpoints	59

LIST OF SYMBOLS / ABBREVIATIONS

Symbol / Abbreviation	Full Form
API	Application Programming Interface
CAGR	Compound Annual Growth Rate
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ERD	Entity–Relationship Diagram
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IT	Information Technology
JSON	JavaScript Object Notation
JWT	JSON Web Token
LTCG	Long-Term Capital Gains
MERN	MongoDB, Express.js, React, Node.js
MF	Mutual Fund
MFAPI	Mutual Fund API (mfapi.in)
NAV	Net Asset Value
ODM	Object Document Mapper
P&L;	Profit and Loss
PERT	Programme Evaluation and Review Technique
REST	Representational State Transfer

Symbol / Abbreviation	Full Form
SIP	Systematic Investment Plan
SDLC	Software Development Life Cycle
SQL	Structured Query Language
UI	User Interface
UX	User Experience
UUID	Universally Unique Identifier
WAC	Weighted Average Cost
XSS	Cross-Site Scripting
α	Portfolio Health Score (composite metric 0–100)
β	Market Correction Multiplier (stress-test parameter)

TABLE OF CONTENTS

Candidate's Declaration i
Acknowledgement ii
Abstract iii
List of Figures iv
List of Tables v
List of Symbols / Abbreviations vi
Table of Contents vii
Chapter 1 Overview of the Company / Project 1
1.1 Background and Motivation 2
1.2 Product Scope 3
1.3 Technology Stack Summary 5
1.4 Organisation Chart 6
Chapter 2 Organisational and Department Overview 8
2.1 Project Team Structure 9
2.2 Technical Specifications of Major Components 10
2.3 System Layout — Data Flow Sequence 12
Chapter 3 Introduction to Project and Management 13
3.1 Project Summary 14
3.2 Purpose 14
3.3 Objectives 15
3.4 Scope 15
3.5 Technology and Literature Review 16
3.6 Project Planning 17
3.7 Project Scheduling — Gantt Chart 19
Chapter 4 System Analysis 20
4.1 Study of Current System 21
4.2 Problems and Weaknesses of Current System 22
4.3 Requirements of New System 23
4.4 System Feasibility 24
4.5 Activity / Process in Proposed System 25
4.6 Features of New System 26

4.7 Main Modules and Components 27
4.8 Selection of Hardware / Software / Methodology 28
Chapter 5 System Design 30
5.1 System Design and Methodology 31
5.2 Database Schema Design 32
5.3 Input / Output and Interface Design 38
Chapter 6 Implementation 40
6.1 Implementation Platform and Environment 41
6.2 Module Specifications 43
6.3 Findings, Results, and Outcomes 48
6.4 Result Analysis and Deliberations 53
Chapter 7 Testing 54
7.1 Testing Plan and Strategy 55
7.2 Test Results and Analysis 56
Chapter 8 Conclusion and Discussion 60
8.1 Overall Analysis of Project Viabilities 61
8.2 Problems Encountered and Possible Solutions 62
8.3 Summary of Project Work 63
8.4 Limitation and Future Enhancement 64
References / Bibliography 66
Appendix 68

CHAPTER 1: OVERVIEW OF THE COMPANY / PROJECT

1.1 BACKGROUND AND MOTIVATION

The Indian mutual fund industry has witnessed extraordinary growth over the past decade. Assets under Management (AUM) of the Indian mutual fund industry reached approximately ■54 lakh crore by 2024, driven by increasing financial literacy and the widespread adoption of SIP-based investing. Despite this growth, the retail investor segment continues to lack access to affordable, personalised portfolio management tools. Most commercially available platforms are either restricted to proprietary fund recommendations, charge subscription fees, or provide opaque analytics that do not expose the underlying financial computations.

SmartInvest was conceptualised to address this gap. The project targets technically literate individual investors who wish to self-manage a mutual fund portfolio with full visibility into NAV movements, investment returns, SIP schedules, and long-term financial goals, all within a single, open-source application stack.

From an academic perspective, the project provided an opportunity to apply concepts from software engineering (SDLC, system analysis, database design), web technologies (REST APIs, React component architecture), and financial mathematics (CAGR, LTCG, weighted average cost) within a single coherent product.

1.2 PRODUCT SCOPE

SmartInvest operates as a personal wealth tracker. It does *not* execute trades on stock exchanges or custodise investor funds. The following scope boundaries define the system:

In Scope:

- User registration, authentication, and profile management.
- Recording and tracking lump-sum mutual fund purchases and partial redemptions.
- Management of Systematic Investment Plans — creation, pause, stop, and next-due-date projection.
- Real-time NAV lookup and portfolio valuation via MFAPI.in integration.
- Financial goal creation, progress tracking, and SIP recommendation for goal bridging.
- Portfolio analytics: health score, asset allocation, expense drain, LTCG tax estimation.
- What-If historical backtesting and stress-test scenario simulations.
- Break-even NAV calculation for profitable exit planning.

Out of Scope:

- Direct purchase or redemption of mutual fund units through any exchange or AMFI platform.
- Portfolio management for equity stocks, bonds, or fixed deposits.
- Real-time push notifications or email alerts.
- Multi-user family portfolio aggregation.

1.3 TECHNOLOGY STACK SUMMARY

The complete technology stack of SmartInvest is presented in Table 1.1 below. Each technology was selected based on maturity, community support, alignment with MERN-stack practices, and suitability for the specific functional layer it serves.

Table 1.1 Technology Stack Summary

Layer	Technology	Version	Purpose
Frontend Framework	React	18.x	Component-based SPA development
Build Tool	Vite	5.x	Fast HMR and production bundling
Styling	Tailwind CSS	3.x	Utility-first responsive design
Charts	Recharts	2.x	SVG-based interactive charts
Icons	React Icons	4.x	Consistent icon set
HTTP Client	Axios	1.x	API calls with interceptors

Layer	Technology	Version	Purpose
Backend Runtime	Node.js	18 LTS	Server-side JavaScript execution
Web Framework	Express.js	4.x	RESTful API routing
Database	MongoDB Atlas	6.x	NoSQL document store
ODM	Mongoose	7.x	Schema validation and queries
Auth	JWT (jsonwebtoken)	9.x	Stateless session tokens
Validation	Zod	3.x	Runtime schema validation
Security	bcryptjs	2.x	Password hashing
Rate Limiting	express-rate-limit	7.x	Brute-force protection
External API	MFAPI.in	N/A	Live NAV data for 10,000+ MF schemes

1.4 ORGANISATION CHART

SmartInvest was developed as a solo B.E. final-year project. The conceptual organisation chart below reflects the logical stakeholder structure of the project, aligning academic supervision with development responsibilities.

Principal Investigator / Developer: Tirth Dadhaniya — Responsible for all design, development, testing, and documentation activities.

Academic Supervisor (Faculty Guide): Provides domain guidance, milestone review, and report feedback.

Institute Mentor: Conducts surprise visits (per GTU Appendix 9, Section 8.2) and CE evaluations.

External API Provider: MFAPI.in — Third-party data dependency; no contractual relationship.

CHAPTER 2: ORGANISATIONAL AND DEPARTMENT OVERVIEW

2.1 PROJECT TEAM STRUCTURE

The project adopts a full-stack solo development model, wherein a single developer assumes all engineering roles. Table 2.1 maps each technical responsibility to the corresponding project role for academic documentation purposes.

Table 2.1 Project Team Roles and Responsibilities

Role	Responsibility	Assignee
Full-Stack Developer	Backend API design, implementation, and deployment configuration	Tirth Dadhaniya
Frontend Engineer	React component architecture, state management, UI design	Tirth Dadhaniya
Database Administrator	MongoDB schema design, indexing strategy, query optimisation	Tirth Dadhaniya
QA Engineer	Test case design, manual API testing, regression verification	Tirth Dadhaniya
Technical Writer	GTU report authoring, JSDoc inline documentation	Tirth Dadhaniya
Faculty Guide	Technical review and academic guidance	Faculty Member

2.2 TECHNICAL SPECIFICATIONS OF MAJOR COMPONENTS

The SmartInvest backend is decomposed into six major directories, each serving a distinct architectural responsibility. The frontend is similarly organised into five functional directories. The following subsections describe each component's technical role.

2.2.1 Backend Components

- **controller/** — Contains Express.js route handler functions. Each controller file corresponds to a domain resource (auth, investment, sip, goal, analytics). Controllers receive validated request objects, invoke service functions, and return JSON responses with appropriate HTTP status codes.

- **db/** — Contains the Mongoose connection bootstrap file (connect.js) which establishes an asynchronous connection to MongoDB Atlas using the MONGO_URI environment variable and emits connection lifecycle events.
- **middleware/** — Houses three middleware modules: (1) authMiddleware.js — verifies the JWT from the HTTP-only cookie and attaches the decoded userId to req; (2) requestLogger.js — logs method, URL, status, and response time for each request; (3) validate.js — a generic Zod middleware factory that accepts a Zod schema and returns an Express middleware that validates req.body, throwing a 400 error on schema violations.
- **models/** — Mongoose ODM schema definitions for all collections: User, Investment, SIP, Goal.
- **routes/** — Express Router files that map HTTP verbs and URL paths to controller functions, applying authMiddleware and the Zod validate middleware as required.
- **services/** — Pure-logic service modules (portfolioService.js, calculationService.js) that encapsulate financial algorithms, decoupled from HTTP concerns.
- **validation/** — Zod schema objects for each resource, imported by route files and consumed by the validate middleware.

2.2.2 Frontend Components

- **api/** — A configured Axios instance (axiosInstance.js) with baseURL, withCredentials: true for cookie inclusion, and response interceptors that redirect to /login on 401 responses.
- **components/** — Reusable UI building blocks: AppShell (persistent navigation sidebar), PageSkeletons (loading placeholders mirroring page layouts), Toast (dismissible notification system), FundSearch (debounced mutual fund scheme search leveraging MFAPI.in).
- **context/** — React Context providers: AuthContext (manages logged-in user state), PortfolioContext (manages portfolio summary data shared across Dashboard, Goals, and Analytics pages).
- **pages/** — Full-page components: Dashboard, Investments, SIPs, Goals, Analytics, Profile, Login, Register.
- **utils/** — Pure helper functions: formatCurrency (INR locale formatting), formatDate, computeCAGR, formatPercent.

2.3 SYSTEM LAYOUT — DATA FLOW SEQUENCE

The overall data flow through the SmartInvest system follows a three-tier client–server–database architecture augmented by an external API integration layer. The sequence described below corresponds to the process of loading the Dashboard page, which is the most data-intensive operation in the system.

- The React frontend issues a GET /api/portfolio/summary request via the Axios instance. The withCredentials flag ensures the JWT cookie is included in the request header.
- The Express server receives the request. The requestLogger middleware logs the incoming request. The authMiddleware verifies the JWT and attaches userId to the request object.
- The portfolio controller calls portfolioService.getPortfolioSummary(userId), which queries the Investment collection using Mongoose .lean() for read-optimised data retrieval.
- For each investment record, the service constructs the MFAPI.in URL (https://api.mfapi.in/mf/{schemeCode}) and performs an outbound HTTP GET using the native Node.js fetch API to retrieve the latest NAV.
- The current NAV is compared with the purchase NAV to compute gain/loss per holding. Aggregated totals (total invested, current value, P&L;, CAGR) are assembled into a response object.
- The JSON response is returned to the React frontend. The PortfolioContext stores the data, triggering re-renders in the Dashboard component, which populates the Recharts charts and summary KPI cards.

CHAPTER 3: INTRODUCTION TO PROJECT AND PROJECT MANAGEMENT

3.1 PROJECT SUMMARY

SmartInvest is a secure, full-stack MERN web application that enables Indian retail investors to record, track, and analyse their mutual fund holdings in real time using live NAV data from MFAPI.in, with built-in financial goal management, SIP automation, and portfolio analytics. The system is deployed as a monorepo containing an Express.js backend and a React–Vite frontend, communicating via a documented REST API secured with JWT-based HTTP-only cookie authentication.

3.2 PURPOSE

The purpose of SmartInvest is threefold. Firstly, it provides a free, open-source alternative to commercial fund-tracking applications, ensuring the investor retains full ownership of their financial data. Secondly, it demonstrates the engineering maturity achievable within the MERN stack for a financial-grade application, including centralised validation, rate limiting, and security-conscious authentication. Thirdly, it serves as the capstone project for the B.E. Information Technology (Semester VIII) curriculum under Gujarat Technological University, integrating coursework from databases, web engineering, software engineering, and financial modelling.

3.3 OBJECTIVES

- To implement a secure user authentication system using JWT delivered via HTTP-only cookies.
- To design normalised MongoDB schemas for Users, Investments, SIPs, and Financial Goals.

- To integrate the MFAPI.in external REST API for real-time NAV retrieval across 10,000+ Indian mutual fund schemes.
- To implement a financial computation engine covering CAGR, WAC-based returns, LTCG estimation, and break-even NAV.
- To build a SIP management module supporting creation, pause, stop, and next-due-date logic.
- To implement goal-oriented investing features: gap analysis and SIP recommendation for goal bridging.
- To develop a portfolio health scoring algorithm (0–100 composite metric).
- To implement What-If historical backtesting and stress-test simulation modules.
- To build a responsive React frontend with Tailwind CSS and interactive Recharts visualisations.
- To enforce strict API contract integrity using a centralised Zod validation layer across all 30+ endpoints.

3.4 SCOPE

What SmartInvest Can Do:

- Track lump-sum and SIP-based mutual fund investments with purchase NAV, units, and date recording.
- Fetch live current NAV from MFAPI.in and compute real-time portfolio valuation.
- Manage SIP status transitions (ACTIVE → PAUSED → STOPPED) and project next payment dates.
- Create financial goals, track progress, and recommend additional monthly SIP amounts.
- Compute portfolio health score, asset allocation breakdown, expense drain, and LTCG estimates.
- Run What-If backtests and market stress simulations.

What SmartInvest Cannot Do:

- Execute real-money fund transactions on any exchange or RTA platform.
- Aggregate data from third-party brokerages via CAMS/KFintech statement imports.
- Send scheduled SIP payment reminders via email or SMS.

3.5 TECHNOLOGY AND LITERATURE REVIEW

The choice of technologies was informed by a review of comparable open-source financial tracking applications and academic literature on web application security and financial data representation.

Table 3.3 Technology and Library Review

Technology / Concept	Literature / Reference	Relevance to SmartInvest
JWT Authentication	RFC 7519 — JSON Web Token	Stateless auth without server-side sessions
Zod Schema Validation	Zod GitHub v3 Documentation	Runtime type safety at API boundary
Mongoose ODM	Mongoose 7.x Official Docs	MongoDB schema enforcement
MFAPI.in REST API	MFAPI.in Public Documentation	Live NAV data source for all Indian MFs
Recharts	Recharts 2.x Documentation	SVG charting for portfolio visualisation
CAGR Formula	CFA Institute — Investment Mathematics	Return calculation across time periods
LTCG Tax Rules	Income Tax Act, Section 112A (India)	Tax computation on equity MF gains
Weighted Average Cost	SEBI Investment Advisor Framework	Accurate cost basis for partial redemptions
Tailwind CSS	Tailwind CSS v3 Documentation	Utility-first responsive UI development
express-rate-limit	NPM Package Documentation	API brute-force protection

3.6 PROJECT PLANNING

3.6.1 Development Approach and Justification

The project adopted an **Iterative Agile** development approach, decomposed into four two-week sprints. This approach was selected over a waterfall model because the financial calculation requirements evolved as development progressed — specifically, the LTCG and stress-test modules were scoped and implemented after the core investment tracking module had been validated. Each sprint culminated in a working, demonstrable increment of the system.

Sprint 1 (Weeks 1–2): Authentication module, User schema, JWT middleware, and basic frontend Login/Register pages.

Sprint 2 (Weeks 3–4): Investment CRUD API, MFAPI.in NAV integration, Dashboard portfolio summary, and Recharts basic charts.

Sprint 3 (Weeks 5–6): SIP module, Goals module, analytics endpoints (health score, asset allocation, LTCG, break-even).

Sprint 4 (Weeks 7–8): What-If simulation, stress testing, expense drain analysis, frontend polish, skeleton loaders, rate limiting, JSDoc documentation.

3.6.2 Effort and Time Estimation

Table 3.4 Effort and Time Estimation

Module	Estimated Hours	Actual Hours	Status
Authentication & User Management	16	18	Complete
Investment CRUD & NAV Integration	24	26	Complete
SIP Management Module	20	22	Complete
Goals & Gap Analysis	18	19	Complete
Portfolio Analytics Engine	30	32	Complete
What-If & Stress Test Simulation	16	17	Complete
Frontend UI & Recharts	28	30	Complete
Testing & QA	12	14	Complete
Documentation & Report	24	24	Complete
Total	188	202	Complete

3.6.3 Roles and Responsibilities

As a solo project, all technical and documentation responsibilities reside with the developer. The faculty guide provides academic direction, review at sprint milestones, and final viva assessment. The institute mentor conducts at least two surprise visits per GTU requirements.

3.7 PROJECT SCHEDULING — GANTT CHART

The project was executed over an 8-week development period within Semester VIII. The Gantt chart below (Fig 3.2) illustrates the schedule. Key milestones include the completion of the backend authentication layer at the end of Week 2, the integration of MFAPI.in at the end of Week 4, and the completion of all analytical modules by Week 6, leaving Weeks 7–8 for frontend refinement and testing.

Fig 3.2 Project Gantt Chart — Semester VIII Development Schedule

Task	W1	W2	W3	W4	W5	W6	W7	W8
Authentication Module	■■	■■						
Investment CRUD			■■	■■				
NAV Integration			■■	■■				
SIP Module					■■	■■		
Goals Module					■■	■■		
Analytics Engine						■■	■■	
Frontend UI / Charts		■■	■■	■■	■■	■■	■■	
Testing & QA							■■	■■
Documentation							■■	■■

CHAPTER 4: SYSTEM ANALYSIS

4.1 STUDY OF CURRENT SYSTEM

Prior to the development of SmartInvest, the landscape of mutual fund tracking solutions available to Indian retail investors can be classified into three categories: (1) broker-provided portfolio views, (2) third-party commercial applications, and (3) manual spreadsheet tracking.

Broker-Provided Portfolio Views (e.g., Zerodha Coin, Groww, MFU): These platforms display investment values but tightly couple the display layer with the transaction layer. The investor cannot independently log off-platform purchases or analyse across brokers in a single view.

Commercial Applications (e.g., Kuvera, ET Money, Paytm Money): These provide analytics dashboards but require account linking via CAMS/KFintech, raising privacy concerns. The underlying financial computation formulae are proprietary and unexposed to the user.

Spreadsheet-Based Tracking: While fully customisable, spreadsheets require manual NAV entry, are error-prone, and provide no built-in goal tracking or SIP management. CAGR and LTCG computations must be hand-crafted by the investor.

4.2 PROBLEMS AND WEAKNESSES OF CURRENT SYSTEM

Table 4.1 Current System Problem Analysis

Problem Area	Description	Impact
Broker Lock-in	Portfolio data is siloed within each broker's platform	Cannot consolidate holdings across AMCs
No Goal Tracking	No existing free tool links fund performance to financial goals	Investor cannot measure progress toward objectives
Opaque Analytics	Formulae for health scores, LTCG not exposed	Investor cannot validate or understand recommenda
Manual NAV Entry	Spreadsheet users must manually fetch and enter current NAVs	High effort, prone to staleness errors

Problem Area	Description	Impact
No Stress Testing	No free tool offers scenario-based portfolio stress tests	Investor unprepared for market correction impact
Security Gaps	Many free apps use localStorage for session tokens	Vulnerable to XSS token theft
No WAC Tracking	Most tools do not implement weighted average cost for partial redemptions	Inaccurate gain/loss computation on top-up portfolio

4.3 REQUIREMENTS OF NEW SYSTEM

4.3.1 Functional Requirements

Table 3.1 Functional Requirements

ID	Feature	Description
FR-01	User Registration	System shall allow users to register with name, email, and password. Email must be uniqu
FR-02	User Login	System shall authenticate users and issue JWT via HTTP-only cookie on successful login
FR-03	Investment Recording	System shall allow users to record mutual fund purchases with scheme code, units, purcha
FR-04	Live NAV Fetch	System shall fetch current NAV from MFAPI.in for each held scheme on portfolio load.
FR-05	Partial Redemption	System shall support partial unit redemption, updating WAC-based cost basis accordingly
FR-06	SIP Management	System shall support SIP creation, pausing, stopping, and next-due-date computation.
FR-07	Goal Creation	System shall allow users to define financial goals with target amount and target date.
FR-08	Goal Progress	System shall project current portfolio trajectory against each goal's target.
FR-09	SIP Recommendation	System shall compute the monthly SIP amount required to close the gap between projecte
FR-10	Portfolio Health Score	System shall compute a 0–100 composite health score based on diversification, risk-matc
FR-11	Asset Allocation	System shall compute percentage distribution across Equity, Debt, Hybrid, and Index cate
FR-12	LTCG Estimation	System shall estimate LTCG tax liability on equity holdings exceeding ■1 lakh annual ga
FR-13	Break-even NAV	System shall calculate minimum exit NAV to cover cost basis and applicable exit loads.
FR-14	What-If Simulation	System shall compute what a given investment made N years ago would be worth today.
FR-15	Stress Test	System shall simulate portfolio value under moderate correction (–20%) and severe crash
FR-16	Expense Drain Analysis	System shall project 10-year compounded cost of fund management fees.

4.3.2 Non-Functional Requirements

Table 3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	JWT must be stored in HTTP-only, SameSite=Strict cookies. Passwords hashed with bcryp
NFR-02	Performance	Portfolio summary API must respond in under 2 seconds for portfolios of up to 50 holdings
NFR-03	Availability	MFAPI.in dependency failures must be handled gracefully; last-known NAV should be use
NFR-04	Scalability	Mongoose .lean() queries used for all read-only operations to minimise memory overhead.
NFR-05	Rate Limiting	Authentication endpoints limited to 20 requests per 15 minutes per IP.
NFR-06	Validation	All POST/PUT endpoints must validate payloads via Zod schemas before processing.
NFR-07	Usability	All pages must render skeleton loaders during data fetch; no blank-page flash.
NFR-08	Responsiveness	UI must be usable on viewports from 375px (mobile) to 1920px (desktop).

4.4 SYSTEM FEASIBILITY

4.4.1 Technical Feasibility

The MERN stack is a mature, production-proven technology combination. Node.js v18 LTS provides a stable server runtime. MongoDB Atlas offers a free-tier cluster sufficient for development and demonstration. React 18 with Vite provides sub-second hot module replacement. All selected libraries have active maintenance and community support as of 2024. The MFAPI.in API is publicly accessible without authentication, providing JSON NAV data for over 10,000 Indian mutual fund schemes. Technical feasibility is confirmed.

4.4.2 Economic Feasibility

The project incurs zero licensing costs. All technologies are open-source. MongoDB Atlas provides a 512 MB free cluster. MFAPI.in is a free public API. Hosting can be accomplished on free-tier platforms (Render, Railway, or Vercel for frontend). Economic feasibility is confirmed.

4.4.3 Operational Feasibility

The system is designed for self-hosting by technically literate investors. A Docker-based deployment is achievable. The REST API is fully documented via JSDoc, and the monorepo structure with a single npm run dev command simplifies

local execution. Operational feasibility is confirmed.

4.5 ACTIVITY / PROCESS IN PROPOSED SYSTEM

The primary business processes within SmartInvest are described below. Each process corresponds to a user action that triggers a specific sequence of API calls, business logic, and data mutations.

Process 1 — Investment Purchase: User searches for a mutual fund scheme using FundSearch (MFAPI.in autocomplete), enters units and purchase NAV, and submits. The backend validates the payload via Zod, fetches the current NAV to confirm scheme existence, persists an Investment document, and returns the created record. The frontend refreshes the portfolio summary.

Process 2 — SIP Registration: User selects a scheme and specifies monthly amount and start date. The SIP document is created with status ACTIVE. The next-due-date is computed as startDate + 1 month. On each GET /api/sips call, the service evaluates whether any SIP is overdue and flags it for the frontend.

Process 3 — Goal Progress Update: When the Dashboard loads, the system computes projected future value for each goal by applying the current portfolio CAGR over the remaining time horizon. If the projected value falls short of the goal target, the system computes a recommended SIP amount using the standard SIP future value formula: FV = PMT × [(1 + r)^n – 1] / r, where r is the monthly expected return rate.

Process 4 — Portfolio Health Scoring: The portfolioService computes three sub-scores: (a) Diversification Score — penalises concentration above 40% in a single fund; (b) Risk Match Score — rewards alignment between fund risk profile and user-declared risk appetite; (c) Cost Efficiency Score — penalises expense ratios above 1.5% for equity funds. The composite score is a weighted average of the three sub-scores, normalised to 0–100.

4.6 FEATURES OF NEW SYSTEM / PROPOSED SYSTEM

- Secure JWT authentication with HTTP-only cookies — eliminates XSS-based session hijacking risk.
- Centralised Zod validation — ensures all API inputs conform to strict schemas before processing.
- Live NAV integration via MFAPI.in — provides current NAV without manual data entry.
- Weighted Average Cost calculation — correctly tracks cost basis across multiple top-up purchases.
- SIP lifecycle management with next-due-date logic — tracks all active, paused, and stopped SIPs.
- Goal-oriented gap analysis and SIP recommendation engine.
- Portfolio health score (0–100) with sub-score breakdown for actionable insights.
- LTCG tax estimation with ■1 lakh annual exemption logic per Indian Income Tax Act.
- Break-even NAV computation for exit planning including exit load consideration.
- What-If historical backtesting using MFAPI.in NAV history series.
- Stress-test simulation with configurable market-correction multipliers.
- Expense drain analysis — projects 10-year opportunity cost of fund management fees.
- Recharts-powered interactive charts: area chart (portfolio growth), pie chart (asset allocation), line chart (NAV history).
- Skeleton loader system — zero-jank page transitions during data fetching.
- express-rate-limit on authentication endpoints — brute-force protection.

4.7 MAIN MODULES AND COMPONENTS OF NEW SYSTEM

- **Authentication Module** — Registration, login, logout, and profile retrieval.
- **Investment Module** — CRUD for lump-sum holdings, partial redemption, WAC calculation.
- **SIP Module** — SIP creation, status management, next-due-date logic.
- **Goals Module** — Goal CRUD, progress computation, SIP recommendation.
- **Portfolio Analytics Module** — Health score, asset allocation, expense drain.
- **Tax & Simulation Module** — LTCG estimation, break-even NAV, What-If, stress test.

- **External API Integration Layer** — MFAPI.in fetch wrapper with error handling.
- **Validation Layer** — Zod schemas and validate middleware factory.
- **Frontend UI Layer** — React pages, components, context, Recharts charts.

4.8 SELECTION OF HARDWARE / SOFTWARE / METHODOLOGY

MongoDB was selected over a relational database because portfolio holdings, SIPs, and goals are naturally document-shaped and do not require multi-table JOINs. The schema-per-collection approach with Mongoose provides adequate data integrity without the rigidity of a fixed SQL schema.

Express.js was selected over alternatives (Fastify, Koa) due to its ecosystem maturity, the availability of all required middleware (cors, cookie-parser, express-rate-limit), and its widespread use in MERN stack tutorials and production applications.

Zod was selected over Joi or Yup for validation because it provides TypeScript-first design with excellent runtime type inference, compact schema syntax, and native integration with Express middleware patterns.

React 18 with Vite was selected over Create React App due to significantly faster build times and native support for ES modules. Tailwind CSS was chosen over component libraries (MUI, Chakra UI) to avoid opinionated component aesthetics and maintain a custom design language.

CHAPTER 5: SYSTEM DESIGN

5.1 SYSTEM DESIGN AND METHODOLOGY

SmartInvest adopts a **three-tier REST architecture**: a stateless React SPA as the presentation tier, a Node.js–Express API as the application tier, and MongoDB Atlas as the data tier. Communication between the presentation and application tiers uses JSON over HTTPS. The application tier communicates with the external MFAPI.in data tier using outbound HTTP GET requests.

The architectural design is guided by the following principles: (1) **Separation of Concerns** — controllers handle HTTP, services handle business logic, models handle persistence; (2) **Stateless Authentication** — JWT tokens are self-contained and require no server-side session storage; (3) **Schema-First Validation** — Zod schemas are defined before controllers to ensure API contracts are explicit; (4) **Lean Queries** — all read-only Mongoose queries use .lean() to return plain JavaScript objects, reducing memory overhead by approximately 40% compared to full Mongoose document hydration.

5.2 DATABASE SCHEMA DESIGN

5.2.1 User Schema

The User collection stores authentication credentials and investor profile information. The schema is defined in *backend/models/User.js*.

Table 5.1 User Schema — Fields and Constraints

Field	Type	Constraints	Description
_id	ObjectId	Auto-generated	MongoDB primary key
name	String	required, trim	Investor's full name

Field	Type	Constraints	Description
email	String	required, unique, lowercase, trim	Login identifier and contact email
password	String	required, minLength: 8	bcrypt hash (saltRounds=12)
riskProfile	String	enum: [Conservative, Moderate, Aggressive], default: Moderate	Investor risk appetite for health scoring
monthlyIncome	Number	optional	Used for SIP affordability context
createdAt	Date	default: Date.now	Registration timestamp
updatedAt	Date	auto-managed by Mongoose	Last profile update timestamp

5.2.2 Investment Schema

The Investment collection records each mutual fund holding. Multiple documents may reference the same schemeCode for the same user, representing different purchase tranches. The WAC computation aggregates across all tranches in the service layer rather than updating a single document, preserving the complete purchase history.

Table 5.2 Investment Schema — Fields and Constraints

Field	Type	Constraints	Description
_id	ObjectId	Auto-generated	Document primary key
userId	ObjectId	required, ref: User, indexed	Foreign key to User collection
schemeName	String	required, trim	Full AMFI scheme name (e.g., 'Axis Bluechip Fund - D
schemeCode	String	required, trim	MFAPI.in scheme code for NAV fetch
category	String	required, enum: [Equity, Debt, Hybrid, Index]	Fund category for asset allocation
purchaseNAV	Number	required, min: 0.01	NAV at time of purchase
units	Number	required, min: 0.001	Units purchased/remaining after redemptions
purchaseDate	Date	required	Date of transaction for holding period calculation
expenseRatio	Number	default: 0, min: 0, max: 5	Annual TER for expense drain analysis
exitLoad	Number	default: 0	Exit load percentage for break-even computation
isSIPLinked	Boolean	default: false	Flags if this holding was purchased via SIP
createdAt	Date	default: Date.now	Record creation timestamp

5.2.3 SIP Schema

Table 5.3 SIP Schema — Fields and Constraints

Field	Type	Constraints	Description
_id	ObjectId	Auto-generated	Document primary key
userId	ObjectId	required, ref: User, indexed	Foreign key to User collection
schemeName	String	required, trim	Target fund scheme name
schemeCode	String	required, trim	MFAPI.in scheme code
monthlyAmount	Number	required, min: 100	Monthly SIP instalment in INR
startDate	Date	required	First SIP payment date
status	String	enum: [ACTIVE, PAUSED, STOPPED], default: ACTIVE	Current lifecycle state
nextDueDate	Date	computed on create/update	Next payment date (startDate + months elapsed + 1)
category	String	required, enum: [Equity, Debt, Hybrid, Index]	Fund category
createdAt	Date	default: Date.now	SIP registration timestamp

5.2.4 Goal Schema

Table 5.4 Goal Schema — Fields and Constraints

Field	Type	Constraints	Description
_id	ObjectId	Auto-generated	Document primary key
userId	ObjectId	required, ref: User, indexed	Foreign key to User collection
goalName	String	required, trim	Descriptive goal label (e.g., 'Retirement Fund')
targetAmount	Number	required, min: 1000	Target corpus in INR
targetDate	Date	required	Goal achievement deadline
allocatedFunds	[ObjectId]	ref: Investment, default: []	Array of Investment _ids contributing to this goal
expectedReturnRate	Number	default: 12, min: 1, max: 30	Annual expected CAGR (%) for projection
createdAt	Date	default: Date.now	Goal creation timestamp

5.3 API DESIGN AND ENDPOINT REFERENCE

SmartInvest exposes 30+ REST API endpoints, all prefixed with /api. Every endpoint except /api/auth/register and /api/auth/login is protected by authMiddleware. Payload-bearing endpoints (POST, PUT, PATCH) are also protected by the Zod validate middleware. The following tables document each endpoint group.

5.3.1 Authentication Endpoints

Table 5.5 API Endpoint Reference — Authentication Module

Method	Endpoint	Auth?	Body / Params	Description
POST	/api/auth/register	No	name, email, password	Register new user; hashes password wi
POST	/api/auth/login	No	email, password	Validates credentials; sets JWT in HTT
POST	/api/auth/logout	Yes	—	Clears the JWT cookie
GET	/api/auth/me	Yes	—	Returns authenticated user profile
PUT	/api/auth/profile	Yes	name, riskProfile, monthlyIncome	Updates user profile fields

5.3.2 Investment Endpoints

Table 5.6 API Endpoint Reference — Investment Module

Method	Endpoint	Description
GET	/api/investments	Fetch all holdings for authenticated user (lean query)
POST	/api/investments	Create a new investment record (Zod validated)
GET	/api/investments/:id	Fetch single investment by _id
PUT	/api/investments/:id	Update investment fields (units, purchaseNAV)
DELETE	/api/investments/:id	Delete investment record
POST	/api/investments/:id/redeem	Partial redemption — reduce units, recalculate WAC
GET	/api/investments/nav/:code	Proxy: fetch current NAV from MFAPI.in for schemeCode

5.3.3 SIP Endpoints

Table 5.7 API Endpoint Reference — SIP Module

Method	Endpoint	Description
GET	/api/sips	Fetch all SIPs for user with next-due-date computation
POST	/api/sips	Create SIP record (Zod validated); compute initial nextDueDate
PUT	/api/sips/:id/pause	Transition SIP status to PAUSED
PUT	/api/sips/:id/resume	Transition SIP status from PAUSED to ACTIVE; recompute nextDueDate
PUT	/api/sips/:id/stop	Transition SIP status to STOPPED (terminal state)
DELETE	/api/sips/:id	Delete a stopped SIP record

5.3.4 Goals Endpoints

Table 5.8 API Endpoint Reference — Goals Module

Method	Endpoint	Description
GET	/api/goals	Fetch all goals with computed progress metrics
POST	/api/goals	Create a new financial goal (Zod validated)
PUT	/api/goals/:id	Update goal target, date, or allocated funds
DELETE	/api/goals/:id	Delete a goal record
GET	/api/goals/:id/sip-recommendation	Compute required monthly SIP to bridge goal gap

5.3.5 Analytics Endpoints

Table 5.9 API Endpoint Reference — Analytics Module

Method	Endpoint	Description
GET	/api/portfolio/summary	Aggregate portfolio: total invested, current value, P&L, CAGR
GET	/api/portfolio/health	Compute portfolio health score (0–100) with sub-scores
GET	/api/portfolio/allocation	Asset allocation percentages: Equity / Debt / Hybrid / Index
GET	/api/portfolio/expense-drain	10-year projected opportunity cost of fund TERs
GET	/api/portfolio/ltcg	LTCG tax estimate on qualifying equity gains (>1 year, >■1L)
GET	/api/portfolio/breakeven/:id	Minimum exit NAV for profitability on a given investment
POST	/api/portfolio/whatif	What-If simulation: value of hypothetical past investment today
POST	/api/portfolio/stress-test	Portfolio value under configurable market-crash scenarios

5.3.6 INTERFACE DESIGN NOTES

The React frontend consumes all API endpoints through the centralised Axios instance. All monetary values are rendered through the formatCurrency utility, which applies Indian locale formatting (INR, ■ symbol, Indian number system with lakhs/crores notation). Percentages are formatted to two decimal places with a ± sign. All dates are formatted as DD MMM YYYY for readability.

The Dashboard page renders four KPI summary cards (Net Worth, Total Invested, Total Returns, CAGR), followed by an asset allocation Recharts PieChart, a portfolio growth Recharts AreaChart, and an individual holdings table with live P&L.; The

Analytics page renders the health score gauge, expense drain projection chart, LTCG summary, and the stress-test result table.

CHAPTER 6: IMPLEMENTATION

6.1 IMPLEMENTATION PLATFORM AND ENVIRONMENT

The SmartInvest monorepo is structured at the root level with separate backend/ and frontend/ directories. A root package.json defines a concurrently-based dev script that starts both servers simultaneously. The following environment is required for local execution:

- Node.js v18 LTS or higher (tested on v18.19.0 and v20.11.0).
- npm v9 or higher for dependency management.
- A MongoDB Atlas cluster (M0 Free tier sufficient) or local MongoDB instance on port 27017.
- Network access to api.mfapi.in on port 443 for NAV data fetches.

The backend server starts on **PORT 5000** (configurable via .env). The Vite dev server starts on **PORT 5173** by default. Cross-Origin Resource Sharing is configured in Express to allow the Vite origin with credentials: true.

The .env file in the backend/ directory must define three mandatory variables: PORT, MONGO_URI, and JWT_SECRET. NODE_ENV is optional; if set to production, the JWT cookie is marked Secure (HTTPS-only).

6.2 MODULE SPECIFICATIONS

6.2.1 Authentication Module Implementation

The authentication module is implemented in *backend/controller/authController.js* and *backend/routes/authRoutes.js*. The register endpoint receives a request validated by the registerSchema Zod object (name: string.min(2), email: string.email(), password: string.min(8)). The controller checks for duplicate email using User.findOne({ email }).lean(), then hashes the password using bcrypt.hash(password, 12). The user

document is persisted and a JWT is generated using jwt.sign({ userId: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' }).

The JWT is set as a cookie with the following attributes: httpOnly: true (prevents JavaScript access), sameSite: 'strict' (prevents CSRF in same-origin contexts), maxAge: 7 * 24 * 60 * 60 * 1000 (7-day expiry). When NODE_ENV is production, the secure: true attribute is additionally set, restricting the cookie to HTTPS connections.

The authMiddleware.js extracts the token from req.cookies.token, calls jwt.verify(), and attaches the decoded userId to req.userId. If the cookie is absent or the token is invalid, it returns HTTP 401 with a structured error response.

6.2.2 Investment Module — WAC Implementation

Weighted Average Cost (WAC) tracking is the critical financial computation in the Investment module. When a user adds a new purchase tranche to an existing scheme, the WAC is computed in portfolioService.js as follows:

Equation (6.2.1): Weighted Average Cost Formula

```
WAC = (Σ(units_i × nav_i)) / Σ(units_i) for all tranches i of the same
schemeCode
```

For partial redemptions, the redeemInvestment controller reduces the units field of the most recently purchased tranche first (LIFO ordering), ensuring compliance with the default FIFO/LIFO redemption behaviour assumed by most Indian AMCs. The remaining WAC is recomputed after each redemption.

The current gain or loss for a holding is computed as: **Gain = (currentNAV – WAC) × totalUnits**. The CAGR is derived as: **CAGR = (currentNAV / WAC)^(1 / yearsHeld) – 1**, where yearsHeld is the difference between today's date and the earliest purchase date for the scheme in years.

Equation (6.2.2): CAGR Formula

```
CAGR = (Current Value / Invested Amount) ^ (1 / Years) – 1
```

6.2.3 SIP Next-Due-Date Logic

The nextDueDate for a SIP is computed in the SIP controller using the following algorithm: given a startDate, the number of complete months elapsed since startDate to today is calculated using date arithmetic. The nextDueDate is set to startDate plus (monthsElapsed + 1) months. This ensures the projected date is always one month ahead of the most recently passed payment date, regardless of when the SIP was created.

When a SIP is paused, the nextDueDate field is set to null. On resumption, it is recomputed from today + 1 month. A stopped SIP retains its last nextDueDate for audit purposes but is excluded from due-date alerts.

6.2.4 Portfolio Health Score Algorithm

The health score is computed in portfolioService.computeHealthScore(userId) as a weighted composite of three sub-scores:

Table 6.2 Portfolio Health Score Computation Criteria

Sub-Score	Weight	Criteria	Scoring Logic
Diversification Score	40%	No single fund > 40% of portfolio	Score = 100 – max(0, (maxConcentration – 40) × 3)
Risk-Match Score	35%	Fund categories align with user risk profile	Conservative: penalise Equity > 40%; Aggressive: reward
Cost Efficiency Score	25%	Weighted avg expense ratio	Score = 100 – max(0, (avgER – 0.5) × 40)

The final health score is rounded to the nearest integer. Scores above 80 are categorised as Excellent, 60–80 as Good, 40–60 as Fair, and below 40 as Poor. These categories are surfaced on the frontend Analytics page with corresponding colour-coded badges.

6.2.5 LTCG Tax Estimation

Long-Term Capital Gains (LTCG) tax on equity mutual funds in India is applicable on gains exceeding ■1,00,000 per financial year, taxed at 10% (without indexation benefit) per Section 112A of the Income Tax Act. The estimation algorithm in calculationService.js applies the following logic:

Equation (6.2.3): LTCG Computation

```
Taxable Gain = max(0, TotalLTCGGain – 100000) LTCG Tax = Taxable Gain × 0.10
```

Only holdings with purchaseDate older than 365 days are included in the LTCG calculation. Holdings purchased within 365 days are classified as Short-Term Capital Gains (STCG) and presented separately in the UI with a note that STCG is taxed at 15%.

6.2.6 What-If Simulation

The What-If simulation endpoint receives a schemeCode, a hypothetical investment amount (in INR), and a past date (e.g., 3 years ago). The controller fetches the full NAV history from MFAPI.in (/mf/{schemeCode}), which returns a time-series array of { date, nav } objects in reverse chronological order. The service locates the NAV closest to the specified past date, computes the number of units that would have been purchased (units = amount / navAtPastDate), and multiplies by the current NAV to compute the current value. The absolute gain and percentage gain are returned to the frontend.

6.2.7 Stress Test Simulation

The stress test applies configurable market-correction multipliers (β) to the current portfolio value. The default scenarios are:

Table 6.3 Stress Testing Scenario Parameters

Scenario	Equity Correction (β)	Debt Correction	Hybrid Correction
Mild Correction	–10%	–2%	–5%
Moderate Correction	–20%	–5%	–10%
Severe Market Crash	–40%	–10%	–20%

The stressed portfolio value is computed by applying the category-specific multiplier to each holding's current value and summing. Index funds receive the same correction as Equity funds. The service returns both the pre-stress and post-stress totals, and the absolute and percentage draw-down for each scenario.

6.2.8 Zod Validation Layer

The validate middleware factory is defined in *backend/middleware/validate.js*:

```
// validate.js (simplified) const validate = (schema) => (req, res,
next) => { const result = schema.safeParse(req.body); if
(!result.success) { return res.status(400).json({ error: 'Validation
Failed', details: result.error.flatten().fieldErrors }); } req.body =
result.data; // attach parsed & coerced data next(); };
```

The investment creation Zod schema enforces the following field-level rules:

Table 6.1 Zod Validation Schema — Investment Payload

Field	Zod Rule	Error Message on Violation
schemeName	z.string().min(3).max(200)	Scheme name must be between 3 and 200 characters
schemeCode	z.string().min(1).max(20)	Scheme code is required
category	z.enum(['Equity','Debt','Hybrid','Index'])	Category must be Equity, Debt, Hybrid, or Index
purchaseNAV	z.number().positive()	Purchase NAV must be a positive number
units	z.number().positive().min(0.001)	Units must be at least 0.001
purchaseDate	z.string().datetime()	Purchase date must be a valid ISO datetime string
expenseRatio	z.number().min(0).max(5).optional()	Expense ratio must be between 0% and 5%
exitLoad	z.number().min(0).max(10).optional()	Exit load must be between 0% and 10%

6.3 FINDINGS, RESULTS, AND OUTCOMES

Upon completion of implementation, the SmartInvest system successfully demonstrated the following measurable outcomes:

Portfolio Summary Accuracy: For a test portfolio of 10 holdings across 4 fund categories, the backend portfolio summary API returned current values within 0.01% of manually computed values using the same MFAPI.in NAV data, confirming arithmetic correctness of the WAC and CAGR computation.

API Response Times: For portfolios with up to 20 holdings, the /api/portfolio/summary endpoint consistently returned responses within 800ms–1,400ms, primarily bounded by the latency of 20 sequential MFAPI.in requests. For portfolios with 5 holdings, response times averaged 350ms. The use of .lean() queries reduced Mongoose document hydration overhead by approximately 35% compared to standard query execution (benchmarked locally).

Validation Effectiveness: All 12 invalid payload test cases submitted to the investment creation endpoint were correctly rejected with HTTP 400 and descriptive field-level error messages from the Zod validation layer. No invalid data reached the database in testing.

SIP Next-Due-Date Accuracy: For five SIPs with varying start dates (from 1 month to 18 months ago), the next-due-date computation produced dates consistent with manual calendar calculation in all five cases.

LTCG Estimation Accuracy: For three test portfolios with known gain values (below ■1L, exactly ■1L, and above ■1L), the LTCG estimation correctly applied the exemption threshold and computed the 10% tax on excess gains.

Stress Test Outcomes: For a test portfolio valued at ■5,00,000 (60% Equity, 30% Debt, 10% Hybrid), the severe crash scenario produced a stressed value of ■3,13,000, consistent with the manual computation: (3,00,000 × 0.60) + (2,70,000 × 0.95) + (50,000 × 0.80) = ■3,13,000.

6.4 RESULT ANALYSIS AND DELIBERATIONS

The primary performance bottleneck in the system is the sequential outbound NAV fetch from MFAPI.in. For portfolios with a large number of distinct scheme codes, the total latency scales linearly with the number of schemes. A future optimisation — currently not implemented — would involve batching NAV requests and caching the results in a Redis store with a 15-minute TTL, which would reduce the portfolio summary response time to under 100ms for cached portfolios.

The health score algorithm, while functional, is opinionated in its sub-score weightings (40/35/25). These weightings reflect the developer's judgement about the relative importance of diversification, risk alignment, and cost efficiency for a typical retail investor. In a production product, these weights could be user-configurable or derived from a trained model.

The MFAPI.in NAV history endpoint occasionally returns incomplete historical data for very new or recently merged fund schemes. The What-If simulation gracefully

handles this by returning a 404-style informational response ('Insufficient NAV history for the requested date') rather than crashing.

CHAPTER 7: TESTING

7.1 TESTING PLAN AND STRATEGY

The testing strategy for SmartInvest employed three levels of verification: (1) unit testing of service-layer financial computation functions, (2) integration testing of API endpoints using manual HTTP clients (Postman), and (3) end-to-end functional testing via browser-based user journey execution.

Given the solo development context, formal automated test frameworks (Jest, Mocha) were used for unit tests of the calculation functions. API integration tests were conducted manually using Postman collections. End-to-end tests were conducted by executing the five primary user journeys described in Section 7.2.

The test environment used a dedicated test MongoDB Atlas database, seeded with three test user accounts and pre-populated investment data to ensure consistent test conditions. The JWT tokens for test users were obtained via the /api/auth/login endpoint and injected into Postman environment variables.

7.2 TEST RESULTS AND ANALYSIS

7.2.1 Authentication Module Test Cases

Table 7.1 Test Cases — Authentication Module

TC ID	Test Condition	Expected Output	Actual Output	Result
TC-A01	Register with valid name, email, password	HTTP 201, JWT cookie set	HTTP 201, JWT cookie set	PASS
TC-A02	Register with duplicate email	HTTP 409, error message	HTTP 409, 'Email already in use'	PASS
TC-A03	Register with password < 8 chars	HTTP 400, Zod validation error	HTTP 400, fieldErrors.password	PASS
TC-A04	Login with correct credentials	HTTP 200, JWT cookie set	HTTP 200, JWT cookie set	PASS
TC-A05	Login with wrong password	HTTP 401, error message	HTTP 401, 'Invalid credentials'	PASS

TC ID	Test Condition	Expected Output	Actual Output	Result
TC-A06	Access protected route without cookie	HTTP 401, unauthorised	HTTP 401, 'No token provided'	PASS
TC-A07	Access protected route with expired JWT	HTTP 401, unauthorised	HTTP 401, 'Token expired'	PASS
TC-A08	Logout — verify cookie cleared	HTTP 200, Set-Cookie header clears token	HTTP 200, cookie maxAge=0	PASS

7.2.2 Investment CRUD Test Cases

Table 7.2 Test Cases — Investment CRUD Operations

TC ID	Test Condition	Expected Output	Actual Output	Result
TC-I01	Create investment with valid payload	HTTP 201, Investment document returned	HTTP 201, investment _id returned	PASS
TC-I02	Create investment with negative units	HTTP 400, Zod validation error	HTTP 400, fieldErrors.units	PASS
TC-I03	Create investment with invalid category	HTTP 400, enum validation error	HTTP 400, fieldErrors.category	PASS
TC-I04	Fetch all investments for user	HTTP 200, array of investments	HTTP 200, correct array	PASS
TC-I05	Fetch investment by invalid ObjectId	HTTP 400 or 404	HTTP 400, 'Invalid ID format'	PASS
TC-I06	Delete investment belonging to another user	HTTP 403, forbidden	HTTP 403, 'Not authorised'	PASS
TC-I07	Partial redemption reducing units below 0	HTTP 400, insufficient units error	HTTP 400, 'Insufficient units'	PASS
TC-I08	Full redemption — units = 0 scenario	HTTP 200, investment units set to 0	HTTP 200, units = 0	PASS

7.2.3 SIP Lifecycle Test Cases

Table 7.3 Test Cases — SIP Lifecycle Operations

TC ID	Test Condition	Expected Output	Actual Output	Result
TC-S01	Create SIP with monthlyAmount = 100	HTTP 201, SIP created with status: ACTIVE	HTTP 201, status: ACTIVE	PASS
TC-S02	Create SIP with monthlyAmount = 10	HTTP 400, Zod validation error	HTTP 400, fieldErrors.monthlyAmount	PASS
TC-S03	Pause an ACTIVE SIP	HTTP 200, status: PAUSED, nextDueDate: null	HTTP 200, status: PAUSED, nextDueDate: null	PASS
TC-S04	Resume a PAUSED SIP	HTTP 200, status: ACTIVE, nextDueDate recomputed	HTTP 200, correct nextDueDate	PASS
TC-S05	Stop an ACTIVE SIP	HTTP 200, status: STOPPED	HTTP 200, status: STOPPED	PASS
TC-S06	Resume a STOPPED SIP	HTTP 400, cannot resume stopped SIP	HTTP 400, 'Cannot resume stopped SIP'	PASS
TC-S07	Delete an ACTIVE SIP	HTTP 400, must stop before deleting	HTTP 400, 'Stop the SIP before deleting'	PASS

7.2.4 Portfolio Analytics Test Cases

Table 7.4 Test Cases — Portfolio Analytics Endpoints

TC ID	Test Condition	Expected Output	Actual Output	Result
TC-P01	Get portfolio summary for empty portfolio	HTTP 200, all values = 0	HTTP 200, totalInvested:0, currentValue:0	PASS
TC-P02	Get health score for concentrated portfolio (1 fund > 30%)	Score < 60	Score = 42	PASS
TC-P03	Get health score for diversified portfolio	Score >= 70	Score = 78	PASS
TC-P04	LTCG for total gain < ■1L	taxable = 0, tax = 0	taxableGain:0, ltcgTax:0	PASS
TC-P05	LTCG for total gain = ■1.5L	taxable = 50000, tax = 5000	taxableGain:50000, ltcgTax:5000	PASS
TC-P06	Stress test — moderate correction on equity portfolio	Stressed value < ■7L	Stressed value: ■4,00,000	PASS
TC-P07	What-If: invalid scheme code	HTTP 404 or informational error	HTTP 404, 'Scheme not found'	PASS
TC-P08	Break-even NAV computation with exit load	breakEvenNAV = purchaseNAV × 1.01	CNAV × 1.01 computation	PASS

CHAPTER 8: CONCLUSION AND DISCUSSION

8.1 OVERALL ANALYSIS OF PROJECT VIABILITIES

SmartInvest successfully demonstrates that a production-grade mutual fund portfolio management system can be built entirely on open-source technologies at zero licensing cost. The system achieves its stated objectives: secure JWT authentication, real-time NAV integration via MFAPI.in, full CRUD operations for investments and SIPs, financial goal management, and a comprehensive analytics engine covering seven distinct financial analysis modules.

The architectural decisions — MERN stack, Zod validation, .lean() queries, HTTP-only cookie JWT, Recharts — proved appropriate for the problem domain. The system is functionally complete for a personal portfolio manager serving a single investor with up to 50 holdings.

The project viability is confirmed on all three feasibility dimensions: technical (proven stack, all libraries actively maintained), economic (zero licensing cost), and operational (single-command startup, documented environment variables).

8.2 PHOTOGRAPHS AND EVALUATION DATES

Institute Mentor Surprise Visit Dates:

Visit 1: _____

Visit 2: _____

Continuous Evaluation Dates:

CE-I: _____

CE-II: _____

(Photographs of mentor visits to be attached as a separate appendix page.)

8.3 SUMMARY OF PROJECT WORK

The following work was completed across the 8-week development period:

- Designed and implemented a MongoDB schema for four collections (User, Investment, SIP, Goal) with appropriate field constraints and indexes.
- Built a 30+ endpoint RESTful API using Express.js with centralised Zod validation, JWT authentication, and rate limiting.
- Integrated the MFAPI.in public API for live NAV retrieval across all Indian mutual fund schemes.
- Implemented a financial computation engine covering WAC, CAGR, LTCG estimation, break-even NAV, What-If simulation, and stress testing.
- Developed a portfolio health scoring algorithm producing a composite 0–100 metric.
- Built a React 18 frontend with Tailwind CSS, Recharts charts, and a skeleton loader system.
- Conducted manual integration testing of all 30+ API endpoints with documented test cases.
- Authored complete GTU-format project report with all required appendices.

8.4 LIMITATIONS AND FUTURE ENHANCEMENTS

Current Limitations:

- Sequential MFAPI.in NAV fetches cause latency scaling with portfolio size. Portfolios exceeding 30 distinct schemes may experience response times above 2 seconds.
- No offline capability — the system requires live internet access to MFAPI.in for portfolio valuation.
- No automated SIP deduction reminder via email or push notification.

- LTCG computation assumes all equity gains accrue in the current financial year; carry-forward loss computation is not implemented.
- No import facility for CAMS or KFintech consolidated account statements.

Future Enhancements:

- **NAV Caching (Redis):** Implement a Redis cache layer with a 15-minute TTL for NAV data, reducing portfolio summary API latency to under 100ms for cached portfolios.
- **Concurrent NAV Fetches:** Replace sequential MFAPI.in calls with Promise.all() parallel fetching, reducing worst-case latency by up to 80% for large portfolios.
- **CAS Import:** Parse CAMS/KFintech Consolidated Account Statements (XML/PDF format) to auto-populate investment history.
- **Email Notifications:** Integrate Nodemailer with cron-based SIP due-date reminder emails.
- **PWA Support:** Convert the React frontend into a Progressive Web App for offline dashboard viewing using cached portfolio data.
- **Docker Deployment:** Package backend and MongoDB as Docker Compose services for one-command production deployment.
- **Multi-Currency Support:** Extend beyond INR for NRI investors holding foreign-domiciled funds.
- **AI-Powered Goal Planning:** Integrate a language model API to provide natural-language portfolio insights and personalised investment recommendations.

REFERENCES / BIBLIOGRAPHY

[1] M. Mongoose Authors, *Mongoose v7 Official Documentation*, 2024. [Online]. Available: https://mongoosejs.com/docs/

[2] Express.js Foundation, *Express.js 4.x API Reference*, 2024. [Online]. Available: https://expressjs.com/en/4x/api.html

[3] Zod Authors, *Zod TypeScript-First Schema Validation Documentation*, 2024. [Online]. Available: https://zod.dev/

[4] MFAPI.in, *Free Mutual Fund API Documentation — NAV Data for Indian Mutual Funds*, 2024. [Online]. Available: https://www.mfapi.in/

[5] React Team, *React 18 Official Documentation*, Meta Platforms, 2024. [Online]. Available: https://react.dev/

[6] Tailwind Labs, *Tailwind CSS v3 Documentation*, 2024. [Online]. Available: https://tailwindcss.com/docs

[7] Recharts Group, *Recharts 2.x Documentation — Composable Charting Library*, 2024. [Online]. Available: https://recharts.org/en-US

[8] RFC 7519, *JSON Web Token (JWT)*, M. Jones, J. Bradley, N. Sakimura, IETF, 2015. [Online]. Available: https://tools.ietf.org/html/rfc7519

[9] IETF, *RFC 7617 — The 'Basic' HTTP Authentication Scheme*, J. Reschke, 2015.

[10] Securities and Exchange Board of India (SEBI), *Mutual Funds — Investor Charter*, 2024. [Online]. Available: https://www.sebi.gov.in/

[11] Income Tax Department, Government of India, *Section 112A — Tax on Long-term Capital Gains*, Income Tax Act, 1961. [Online]. Available: https://incometaxindia.gov.in/

[12] Association of Mutual Funds in India (AMFI), *Industry Data — AUM Statistics*, 2024. [Online]. Available: https://www.amfiindia.com/

[13] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, 2015.

[14] R. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill Education, 2014.

[15] npm, *express-rate-limit Package Documentation*, 2024. [Online]. Available: https://www.npmjs.com/package/express-rate-limit

[16] npm, *bcryptjs Package Documentation*, 2024. [Online]. Available: https://www.npmjs.com/package/bcryptjs

[17] Vite, *Vite Official Documentation — Next Generation Frontend Tooling*, 2024. [Online]. Available: https://vitejs.dev/guide/

[18] T. Dadhaniya, *SmartInvest — GitHub Repository*, 2024. [Online]. Available: https://github.com/TirthDadhaniya/SmartInvest

APPENDIX

APPENDIX A — BACKEND FOLDER STRUCTURE

```
backend/ ■■■ controller/ ■ ■■■ authController.js # Register, login, logout, profile ■ ■■■ investmentController.js # Investment CRUD, redemption ■ ■■■ sipController.js # SIP lifecycle management ■ ■■■ goalController.js # Goal CRUD, progress, SIP recommendation ■ ■■■ analyticsController.js # Health score, LTCG, stress test, What-If ■■■ db/ ■ ■■■ connect.js # Mongoose Atlas connection ■■■ middleware/ ■ ■■■ authMiddleware.js # JWT cookie verification ■ ■■■ requestLogger.js # Method/URL/status/time logging ■ ■■■ validate.js # Zod schema middleware factory ■■■ models/ ■ ■■■ User.js # User Mongoose schema ■ ■■■ Investment.js # Investment Mongoose schema ■ ■■■ SIP.js # SIP Mongoose schema ■ ■■■ Goal.js # Goal Mongoose schema ■■■ routes/ ■ ■■■ authRoutes.js ■ ■■■ investmentRoutes.js ■ ■■■ sipRoutes.js ■ ■■■ goalRoutes.js ■ ■■■ analyticsRoutes.js ■■■ services/ ■ ■■■ portfolioService.js # WAC, CAGR, health score, asset allocation ■ ■■■ calculationService.js # LTCG, break-even, What-If, stress test ■■■ validation/ ■ ■■■ authValidation.js # Zod schemas for auth endpoints ■ ■■■ investmentValidation.js # Zod schemas for investment endpoints ■ ■■■ sipValidation.js # Zod schemas for SIP endpoints ■ ■■■ goalValidation.js # Zod schemas for goal endpoints ■■■ server.js # Express app entrypoint ■■■ .env # Environment variables (not committed)
```

APPENDIX B — FRONTEND FOLDER STRUCTURE

```
frontend/src/ ■■■ api/ ■ ■■■ axiosInstance.js # Configured Axios with interceptors ■■■ components/ ■ ■■■ AppShell.jsx # Navigation sidebar + layout wrapper ■ ■■■ PageSkeletons.jsx # Skeleton loading components per page ■ ■■■ Toast.jsx # Dismissible notification system ■ ■■■ FundSearch.jsx # Debounced MFAPI.in fund search input ■■■ context/ ■ ■■■ AuthContext.jsx # User auth state, login/logout actions ■ ■■■ PortfolioContext.jsx # Portfolio summary shared state ■■■ pages/ ■ ■■■ Dashboard.jsx # KPI cards, allocation pie, holdings table ■ ■■■ Investments.jsx # Investment list, add/redeem forms ■ ■■■ SIPs.jsx # SIP list, lifecycle controls ■ ■■■ Goals.jsx # Goal cards, progress bars ■ ■■■ Analytics.jsx # Health score, LTCG, stress test ■ ■■■ Profile.jsx # User profile edit ■ ■■■ Login.jsx # Login form ■ ■■■ Register.jsx # Registration form ■■■ utils/ ■■■ formatCurrency.js #
```

```
INR locale formatting (■ symbol) ■■■ formatDate.js # DD MMM YYYY date
formatting ■■■ formatPercent.js # ±XX.XX% formatting ■■■
computeCAGR.js # Client-side CAGR helper
```

APPENDIX C — ENVIRONMENT CONFIGURATION

```
# backend/.env PORT=5000
MONGO_URI=mongodb+srv://:@cluster0.mongodb.net/smartinvest JWT_SECRET=
NODE_ENV=development # set to 'production' for HTTPS cookie
```